

N+1문제점이란?

N+1 문제는 Spring Data JPA와 같은 ORM(Object-Relational Mapping) 프레임워크에서 발생할 수 있는 **성능 문제로, 데이터베이스 쿼리의 수가 불필요하게 증가하는 현상을 가리킨다**. 이 문제는 다음과 같은 방식으로 발생한다.

예를 들어,

우리가 실습하고 있는 FreeBoard와 Reply 프로젝트에서 전체게시물을 조회 할 때 게시물마다 **댓글 수를 함께 출력** 하고 싶다고 가정 해 보자.

자유게시판					
번호	제목	작성자	날짜	조회수	
90	제목45 / Reply count : 3	User45	2024-10-31T20:00:43.205723	3	
89	제목44 / Reply count : 1	User44	2024-10-31T20:00:43.204726	2	
88	제목43 / Reply count : 2	User43	2024-10-31T20:00:43.202733	1	
87	제목42 / Reply count : 0	User42	2024-10-31T20:00:43.201736	0	
86	제목41 / Reply count : 0	User41	2024-10-31T20:00:43.200739	0	

FreeBoard Entity에는 Reply Entity를 @OneToMany로 선언 되어 있다.

이때 fetch전략은 지연로딩으로 별도의 선언을 하지 않아도 fetch=FectchType.LAZY 이다.

Service에서 spring data Jpa가 제공하는 전체검색 findAll() 메소드를 호출 한다.

```

public class FreeBoard { //db에 free_board
    @Id //pk
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "free_bno_seq")
    @SequenceGenerator(name = "free_bno_seq", allocationSize = 1, sequenceName = "free_bno_seq")
    private Long bno;
    private String subject;
    private String writer;

    @Column(length = 500)
    private String content;
    private String password;
    private int readnum; //조회수

    @CreationTimestamp
    private LocalDateTime insertDate; //등록일

    @UpdateTimestamp
    private LocalDateTime updateDate; //수정

    @OneToMany(mappedBy = "freeBoard", cascade = CascadeType.ALL, fetch = FetchType.LAZY) // 참조
    // @OneToMany(mappedBy = "freeBoard") // 지연로딩
    private List<Reply> repliesList;

```

선언하지 않아도 기본값

↓

fetch = FetchType.LAZY

```

**/
no usages new *
@Override
public List<FreeBoard> selectAll() {
    return freeBoardRepository.findAll();
}

```

Spring Data JPA에서 제공하는 메소드

return freeBoardRepository.findAll();

```

Hibernate: select fb1_0.bno,fb1_0.content,fb1_0.insert_date,fb1_0.password,fb1_0.readnum,fb1_0.subject,fb1_0.update_date,fb1_0.writer from free_board fb1_0
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?
Hibernate: select r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date from reply r1_0 where r1_0.free_bno=?

```

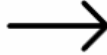
→ 1개

현재 게시물이 10개 이기 때문에 10개의 추가 쿼리가 발생(N+1문제)

→ N개의 쿼리 발생

자유게시판		
번호	제목	작성자
81	제목36 / Reply count : 0	User36
82	제목37 / Reply count : 0	User37
83	제목38 / Reply count : 0	User38
84	제목39 / Reply count : 0	User39
85	제목40 / Reply count : 0	User40
86	제목41 / Reply count : 0	User41
87	제목42 / Reply count : 0	User42
88	제목43 / Reply count : 2	User43
89	제목44 / Reply count : 1	User44
90	제목45 / Reply count : 3	User45

10개
게시물



위의 실행 결과 쿼리를 보면

각 Freeboard 엔티티에 대해 연관된 엔티티(예: Reply 엔티티)를 조회하기 위해 추가 쿼리가 실행된다. 예를 들어, 각 게시물이 10개라면 10개의 추가 쿼리가 실행된다.

fetch = FetchType.EAGER 로 변경을 하여도 N+1 문제는 해결되지 않는다.

JPA는 기본적으로 각 엔티티에 대한 상태를 관리하기 위해 여러 쿼리를 생성할 수 있다. Eager Fetching이 필요한 경우라도 최적화되지 않은 쿼리가 생성될 수 있다. 또한, EAGER는 모든 연관된 데이터를 항상 가져오기 때문에 불필요한 데이터를 로드 할 수 있어 권장되지 않는다.

해결방안은

- 1) Join fetch
- 2) Batch Fetching
- 3) Entity Graph
- 4) QueryDSL

등 다양한 방법이 있다.

일반적으로 가장 많이 사용하는 join fetch를 이용하면 아래와 같다.

XxxRepository interface에 메소드를 정의하고 @Query()를 이용하여 JPQL문법으로 작성한다.

```
usage new
@Query(value = "select f from FreeBoard f left join fetch f.repliesList")
List<FreeBoard> join01();
```

@Override

```
public List<FreeBoard> selectAll() {
    ////fetch조인 사용하기 //////////////////////////////////////
    return freeBoardRepository.join01();
}
```

Hibernate:

```
select fb1_0.bno,fb1_0.content,fb1_0.insert_date,fb1_0.password,fb1_0.readnum,r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date,fb1_0.subject,fb1_0.update_date,fb1_0.writer
from free_board fb1_0 left join reply r1_0 on fb1_0.bno=r1_0.free_bno
```

User_Board_Reply 프로젝트에서 페이징 처리를 위한 JPQL @Query 작성

```
@Query(value = "select distinct f from FreeBoard f left join fetch f.repliesList",
        countQuery = "select count(distinct f.bno) from FreeBoard f left join f.repliesList" )
Page<FreeBoard> join02(Pageable pageable);
```

@Test

```
void join02() {
    Pageable pageable = PageRequest.of(1,2 , Sort.Direction.DESC , "bno");
```

```
//countQuery 적용
Page<FreeBoard> page = freeRep.join02(pageable);

System.out.println("*****");
System.out.println("page.getNumber() = "+page.getNumber());
System.out.println("page.getSize() = "+page.getSize());
System.out.println("page.getTotalPages() = "+page.getTotalPages());
System.out.println("page.previousPageable() = "+page.previousPageable());
System.out.println("page.nextPageable() = "+page.nextPageable());

System.out.println("page.isFirst() = "+page.isFirst());
System.out.println("page.isLast() = "+page.isLast());

System.out.println("page.hasPrevious() = "+page.hasPrevious());
System.out.println("page.hasNext() = "+page.hasNext());
System.out.println("*****");

System.out.println("list.size = " + page.getContent().size());
//list.forEach(b->System.out.println(b.getBno() + " = " + b.getReplyList().size()));

page.getContent().forEach(b->{
    System.out.println(b.getBno()+" | " + b.getSubject());
    b.getReplyList().forEach(r->{
        System.out.println("===> " +r.getRno()+" | " +r.getContent()+" | "+ r.getRno());
    });
    System.out.println();
});
}
```

```

Hibernate: select distinct fb1_0.bno,fb1_0.content,fb1_0.insert_date,fb1_0.password,fb1_0.readnum,r1_0.free_bno,r1_0.rno,r1_0.content,r1_0.insert_date,fb1_0.subject,fb1_0.update_date,fb1_0.writer
from free_board fb1_0 left join reply r1_0 on fb1_0.bno=r1_0.free_bno order by fb1_0.bno desc
|
Hibernate: select count(distinct fb1_0.bno) from free_board fb1_0 left join reply r1_0 on fb1_0.bno=r1_0.free_bno
*****
page.getNumber() = 1
page.getSize() = 2
page.getTotalPages() = 5
page.previousPageable() = Page request [number: 0, size 2, sort: bno: DESC]
page.nextPageable() = Page request [number: 2, size 2, sort: bno: DESC]
page.isFirst() = false
page.isLast() = false
page.hasPrevious() = true
page.hasNext() = true
*****
list.size = 2
88 | 제목43
====> 21 | 안녕 | 21
====> 22 | 하하 | 22

87 | 제목42
```